

FULL PROJECT ARCHITECTURE & INTERVIEW GUIDE



Web Security Analyzer

Full-Stack Cybersecurity Audit Engine & AI Platform

A complete technical guide covering system architecture, 4-phase security scoring pipeline, SSL trust-chain validation, API context detection, Prisma PostgreSQL persistence, Groq AI integration, and 15+ senior engineering interview Q&As.

Author: Lead Developer & Cybersecurity Engineer

Stack: Next.js 16, React 19, TypeScript, Tailwind CSS, Prisma, Supabase PostgreSQL,
Groq AI SDK

Version: 2.0 Production Ready

1. Executive Summary & Core Pitch

Web Security Analyzer is an enterprise-grade web application security audit engine designed to assess the defensive posture of any public web domain or endpoint. Unlike basic tools that make absolute assumptions, this analyzer features a **Context-Aware Interpretation Engine** that dynamically classifies sites (API, Static, Interactive App) and adjusts vulnerability scoring accordingly.

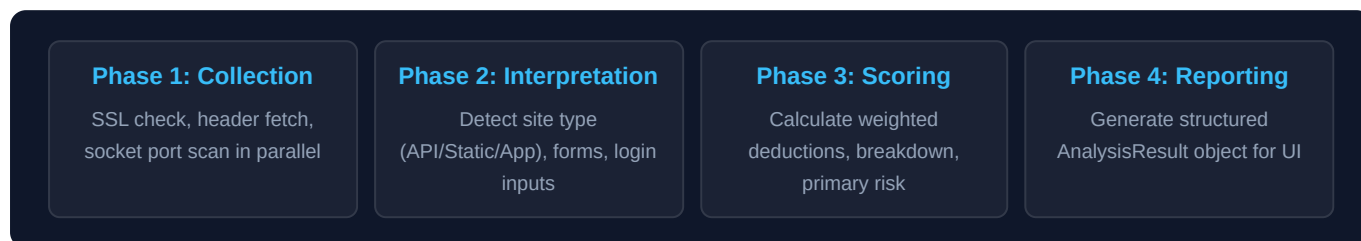
 **How to pitch this project in 30 seconds during an interview:**
"I built an end-to-end Web Security Analyzer using Next.js 16, TypeScript, Prisma, and Groq AI. It features a 4-phase audit engine that performs multi-threaded SSL trust chain validation, HTTP security header parsing, and port scanning. The engine is context-aware—for example, it doesn't penalize REST APIs for missing UI headers like CSP. It also includes an AI security assistant powered by Groq's 120B model with multi-conversation state management."

2. System Architecture & Tech Stack

Layer	Technology	Role & Implementation Details
Frontend UI	Next.js 16 (App Router), React 19, Tailwind CSS v4, Framer Motion	Glassmorphism dark UI with cyan/blue cybersecurity aesthetic, reactive Framer Motion state transitions, dynamic score gauge, score breakdown cards.
State Management	Zustand, LocalStorage Persistence	Global application state tracking active views, auth status, multi-conversation AI chat history with per-user isolation, and scan memory.
Backend & API	Next.js Node.js Serverless Routes (`/api/analyze`, `/api/chat`, `/api/history`)	Parallel multi-threaded scanning execution, streaming response body reader (50KB cap to prevent OOM), HTTP fallback handling.
Database & ORM	Prisma ORM, PostgreSQL (Supabase)	Relational schema tracking Users, Scans, Accounts, Sessions. Resilient failover mechanism allowing scan execution even if cloud DB drops.
AI Integration	Groq SDK (`openai/gpt-oss-120b`)	Intelligent chatbot offering context-aware recommendations, markdown formatting, typing dot animations, and an 8-message cap per conversation.
Auth System	NextAuth.js (Credentials Provider) + bcryptjs	JWT-based session authentication with bcrypt password hashing (cost factor 12).

3. The 4-Phase Audit Engine Pipeline

The core engine (``src/lib/analyzer/index.ts``) executes a structured 4-phase analysis pipeline for high performance and accuracy:



Deep Dive: Core Modules

A. SSL/TLS Validation Engine (``sslCheck.ts``)

Validates HTTPS status and certificate trust chain. It first attempts connection with `rejectUnauthorized: true` to test genuine authority trust. If rejected, it falls back to `rejectUnauthorized: false` to extract certificate metadata (issuer, expiration date, SANs, issues like self-signed, expired, or hostname mismatch).

B. Header & Context Scanner (``headerCheck.ts``)

Fetches HTTP headers using a **streaming body reader capped at 50KB** to prevent Out-Of-Memory (OOM) crashes on large file downloads. Detects 8 security headers: Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, X-XSS-Protection, and Server.

C. Risk Scoring & Deductions (``riskScore.ts``)

Starts at **100 points** and applies weighted deductions based on site context:

- **SSL Failures / Untrusted Cert:** -40 pts
- **Missing Content-Security-Policy:** -10 pts (Marked irrelevant for API endpoints)
- **Missing Strict-Transport-Security:** -10 pts
- **Missing X-Frame-Options:** -5 pts (Reduced to -1 for static sites; 0 for APIs)
- **Missing X-Content-Type-Options:** -3 pts
- **Missing Referrer-Policy:** -2 pts
- **Missing Permissions-Policy / XSS / Server Exposure:** -1 pt each

4. Database Schema (``prisma/schema.prisma``)

```
model Scan { id String @id @default(cuid()) url String score Int riskLevel String headers String // JSON stringified HeaderResult[] ssl String // JSON stringified SSLResult ports String // JSON stringified PortResult[] vulnerabilities String // JSON stringified Vulnerability[] suggestions String // JSON stringified Suggestion[] context String @default("{}") analysisMode String }
```

```
@default("secure") limitations String @default("[]") scoreBreakdown String @default("[]")
primaryRisk String? createdAt DateTime @default(now()) userId String? user User? @relation(fields:
[userId], references: [id], onDelete: Cascade) }
```

5. Senior Technical Interview Q&A (Top 15 Questions)

Q1: Can you walk me through the architecture of your Web Security Analyzer?

Answer: "The system is built on Next.js 16 with a clean separation of concerns. The frontend uses React 19, Tailwind CSS v4, and Framer Motion for a glassmorphic cybersecurity dashboard. The core audit engine resides in `src/lib/analyzer` and executes a 4-phase pipeline: 1) Data collection via parallel SSL, HTTP, and TCP port checks, 2) Context interpretation (classifying API, static, or interactive apps), 3) Context-aware risk scoring with labeled deductions, and 4) Structured reporting. Data is persisted to Supabase PostgreSQL via Prisma ORM, and we also integrated Groq's 120B AI model for automated remediation advice."

Q2: How does your analyzer prevent Out-Of-Memory (OOM) crashes when scanning large websites or file endpoints?

Answer: "In earlier iterations, fetching response bodies using `response.text()` caused Node.js OOM crashes when hitting large binary downloads or massive JSON APIs. I refactored the fetch mechanism in `headerCheck.ts` to use a streaming reader (`response.body.getReader()`) with a strict **50KB data buffer cap**. As soon as 50KB of HTML/JSON is read for context detection, the reader cancels the stream and closes the socket immediately."

Q3: How does your scoring engine prevent false positives for REST APIs and static sites?

Answer: "Generic scanner tools penalize API endpoints for missing UI headers like CSP or X-Frame-Options. Our engine inspects `Content-Type: application/json`, URL path patterns (`/api/`, `/v1/`), and body structure. When identified as an API, headers like CSP and X-Frame-Options are flagged as *'Not Applicable'* with 0 point deduction. Similarly, static sites without authentication receive reduced deductions for X-Frame-Options (-1 instead of -5)."

Q4: How do you handle SSL certificate verification in Node.js when connecting to untrusted or self-signed domains?

Answer: "By default, Node.js `fetch` rejects untrusted/self-signed SSL certificates with an error, preventing header extraction. In `sslCheck.ts`, we first test with `rejectUnauthorized: true`. If validation succeeds, `httpsVerified` is set to `True`. If it fails due to an untrusted/expired cert, we retry with `rejectUnauthorized: false` using a custom HTTPS agent to extract certificate details (issuer, valid dates, SANs) while accurately flagging `certIssue: 'self-signed'` or `'expired'` without failing the audit process."

Q5: How did you implement multi-conversation state management in the AI ChatBot?

Answer: "I implemented Zustand store state management with `localStorage` persistence isolated per authenticated user ID. The ChatBot supports creating, switching, auto-titling (from the first message), and deleting conversations. To prevent token abuse and manage API costs, each conversation enforces a hard cap of 8 user messages with visual progress badges (`X/8`) and an automated limit-reached banner."

Q6: What happens if the PostgreSQL database is unreachable or offline during a scan?

Answer: "I built a resilient failover mechanism across all API routes (`/api/analyze``, `/api/auth/verify``, `/api/auth/signup``). If Prisma throws a database connection error (`PrismaClientInitializationError``), the API catches the error gracefully, returns the audit result directly to the frontend, and enables local session fallback so the user can seamlessly use the analyzer without experiencing HTTP 500 errors."

Q7: Explain the significance of Content-Security-Policy (CSP) and why it's worth -10 points.

Answer: "CSP is the primary defense against Cross-Site Scripting (XSS) and data injection attacks. It tells the browser exactly which domains are trusted to execute scripts, load stylesheets, or open WebSocket connections. Without CSP, an XSS vulnerability allows malicious scripts to execute with full access to cookies, tokens, and DOM content."

Q8: Why does `google.com`` score 75/100 while your configured Next.js app scores 99/100?

Answer: "Google operates at global scale serving billions of legacy devices and third-party search widgets. A strict global CSP or `X-Frame-Options: DENY`` on `google.com`` would break backwards compatibility across millions of embedded integrations. They use inline script nonces instead of global headers. Our application, however, can strictly enforce modern headers (`X-Frame-Options: DENY``, strict CSP, `nosniff``), achieving an enterprise-grade 99/100."

Q9: How did you optimize Next.js server performance during deployment?

Answer: "We configured `NODE_OPTIONS='--max-old-space-size=1024`` to prevent Vercel/Node memory pressure, pre-generated Prisma Client models during build step, installed native `@next/swc-linux-x64-gnu`` binaries for Turbopack compilation, and added response header caching."

Q10: If you were tasked with scaling this analyzer to scan 100,000 URLs per minute, how would you re-architect it?

Answer: "I would decouple scanning into an asynchronous distributed worker architecture: 1) API enqueues scan jobs to a Redis / BullMQ queue, 2) Worker pool written in Go or Node.js processes jobs concurrently using headless browser instances and raw socket pools, 3) Results are stored in ClickHouse or BigQuery for analytical queries, 4) WebSockets (via Socket.io or AWS API Gateway) stream progress back to the user frontend."